Quality Metric

Quality metric (chỉ số chất lượng) là **các tiêu chí đo lường cụ thể** dùng để đánh giá **chất lượng của phần mềm hoặc mã nguồn** trong quá trình phát triển. Các chỉ số này giúp đảm bảo rằng sản phẩm phần mềm được viết ra:

- dễ bảo trì (maintainable)
- ít lỗi (reliable)
- dễ mở rộng (extensible)
- dễ hiểu (readable)
- hiệu suất tốt (efficient)

Một số ví dụ về Quality Metrics phổ biến:

Chỉ số	Giải thích
Code Coverage	Tỷ lệ phần trăm mã nguồn được bao phủ bởi các test case (unit test/integration test).
Cyclomatic Complexity	Đo độ phức tạp logic của một đoạn code (nhiều nhánh if/else/switch làm tăng chỉ số này).
Cohesion	Đo mức độ các thành phần trong một module liên quan đến cùng một nhiệm vụ. Cohesion cao là tốt.
Coupling	Đo mức độ phụ thuộc giữa các module. Coupling thấp là tốt (module hoạt động độc lập).
Defect Density	Số lượng lỗi/bugs trên mỗi 1.000 dòng code (KLOC).
Response Time	Thời gian hệ thống phản hồi với yêu cầu của người dùng.
Mean Time to Failure (MTTF)	Thời gian trung bình giữa 2 lần hệ thống bị lỗi.
Maintainability Index	Chỉ số tổng hợp đo mức độ dễ bảo trì của phần mềm.

🔍Ví dụ:

Question 5: List and describe at least 5 quality metrics to measure code and design quality.

In the development of the USC (Ultrasound Software Control) system, measuring software quality is essential to ensure that the system is maintainable, reliable, secure, and meets medical industry standards. Below are 5 quality metrics that will be applied during the project:

---

1. Cyclomatic Complexity

- **Purpose:** Measures the number of linearly independent paths through the program's source code.
- **Significance:** A higher value indicates more complex logic, which increases the risk of errors and makes the code harder to test and maintain.
- **Application:** Used to identify methods or functions that require refactoring to simplify logic and improve testability.

---

2. Code Coverage

- **Purpose:** Measures the percentage of code that is exercised by automated tests.
- **Significance:** Higher code coverage indicates better-tested code and reduces the chances of undiscovered bugs.
- **Application:** Applied in unit testing and integration testing phases to ensure comprehensive test coverage of the codebase.

---

3. Cohesion

- **Purpose:** Measures how closely related the functions within a single module are.
- **Significance:** High cohesion indicates that a module is focused and performs a single task well, which makes it easier to maintain and reuse.
- **Application:** Assessed during system design to ensure each module has a clear, unified responsibility.

---

4. Coupling

- **Purpose:** Measures the degree of interdependence between software modules.
- **Significance:** Low coupling is desirable as it makes modules more independent and flexible, reducing the ripple effect of changes.
- **Application:** Evaluated when designing system architecture to ensure loose coupling between components.

---

5. Defect Density

- **Purpose:** Measures the number of defects per 1,000 lines of code (KLOC).
- **Significance:** A lower defect density means higher software quality and fewer bugs in production.
- **Application:** Used after testing phases to analyze the stability and reliability of the software.